



به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر
هوش مصنوعی قابل اعتماد

تمرین دوم

نام و نام خانوادگی	مهدی مختاری
شماره دانشجویی	810101515
تاریخ ارسال گزارش	405/2/30

فهرست

3.....	سؤال اول: تفسیرپذیری
3.....	بخش اول: سوالات توضیحی
7.....	بخش دوم: سوالات پیاده سازی
14.....	سوال دوم: توصیف پذیری مدل های هوش مصنوعی با استفاده Grad-CAM
14.....	بخش اول: Grad-CAM
18.....	بخش دوم: FinerGrad-CAM
21.....	بخش سوم: مقایسه
24.....	مراجع

سؤال اول: تفسیر پذیری

بخش اول: سوالات توضیحی

زیربخش اول: PDP

الف) در مواقعی که همبستگی (correlation) زیادی بین ویژگی های selected و complement وجود دارد ما یعنی این دو نسبت به هم یکسری شرایطی و شروطی دارند زیرا از هم مستقل نیستند و این یعنی ممکن است یکسری حالاتی که ما مجبور میکنیم مدل را به ازای تمامی x_c ها در دیتاست حتما تمامی x_s ها را ببیند این موجب ایراد میشود زیرا لزوما هر ترکیب x_s یی با هر x_c یی در واقعیت به وجود نمی آید. همین موضوع موجب میشود ما میزان اهمیت غیرواقعی از x_s بدست آوریم زیرا در حالاتی آن را بررسی کردیم که اصلا در دنیای واقعی برای آن جنس دیتا پیش نمی آید پس در نتیجه تفسیر ما از ویژگی مدل نیز غلط خواهد بود. (مدل ما عملا در آن نقاط داده ای ندیده است که بخواهد حالا پیش بینی خوبی داشته باشد که ما آن را فورس میکنید که بکند).

ب) مشکل PDP را همانطور که در بخش الف گفتیم در این مقاله آمده حل کند. ابتدا مدل f_M را پیشنهاد میدهد که عملا به جای استفاده از marginal density که می آید روی x_1 تمام مقادیر x_2 حتی آنهایی که در دیتاست نیامدند و (با x_1 همه مقادیر نمی آیند چون مستقل نیستند) آمده از روش conditional density استفاده کرده که این امر موجب میشود صرفا در مواقعی $f(x_1, x_2)$ محاسبه بشود که در واقع به ازای آن x_1 ما x_2 هایی هم در دیتاست داریم. اما این روش درست است که مسعله extrapolation را حل میکند. اما مشکل دیگری خودش دارد آن هم این است که به صورت کامل ویژگی x_1 را بررسی نمیکند و اندکی نتایج بدست آمده برای x_1 به x_2 نیز وابستگی دارند. حال برای حل جفت این مشکل ها مقاله روش ALE را پیشنهاد میدهد. روش ALE به این صورت عمل میکند که می آید accumulated local effect ها را بررسی میکند اما فقط در ناحیه ای از x_1 که ما درواقع مقادیری برای x_2 داریم. به صورت کلی روش آن به 3 بخش تقسیم میشود:

ابتدا local effect (derivatives) ها را محاسبه میکند. که این مورد میگوید به ازای تغییر اندکی در x_1 با فرض ثابت نگه داشتن مقدار x_2 چه تغییری میکند کل $f(x_1, x_2)$.

$$f^1(x_1, x_2) \equiv \frac{\partial f(x_1, x_2)}{\partial x_1}$$

شکل 1. Local effect of ALE

سپس میانگین local effect ها را روی مقادیری از x_2 که درواقع همراه آن x_1 می آیند ایندازه میگیرد.

$$\int p_{2|1}(x_2|z_1) f^1(z_1, x_2) dx_2$$

شکل 2. Average local effects using conditional density

و در نهایت هم accumulate میکنیم این avg local effect ها را از x_1 -min تا x_1 .

$$\int_{x_{\min,1}}^{x_1} \int p_{2|1}(x_2|z_1) f^1(z_1, x_2) dx_2 dz_1 - \text{constant},$$

شکل 3. ALE objective function

چون این روش از conditional density استفاده میکند دیگر مشکل extrapolation را نمیخوریم و همچنین چون از مشتق استفاده میکند روی f برحسب x_1 این باعث میشود تغییراتی که f میکند فقط برحسب تغییر x_1 محاسبه بشود و این میزان وابستگی نتیجه تفسیر بر x_2 را از بین میبرد.

(ج) خیر - زیرا خیلی مواقع یک ویژگی به تنهایی استفاده نمیشود و این به همین خاطر است که PDP آن به صورت یک خط افقی شده. اما اگر آن را با یک متغیر دیگری باهم جز x-selected بگذاریم ممکن است کاملاً یک نمودار معناداری مشاهده کنیم و این یعنی این ویژگی استفاده شده است ولی حتماً تأثیر آن همراه با یک ویژگی دیگری دیده میشود. به صورت کلی یک ویژگی میتواند به تنهایی تأثیر گذار نباشد اما درکل تأثیر گذار باشد زیرا ترکیب آن با ویژگی دیگری است که برای مدل مهم بوده فلذا با صرفاً خط افقی بودن یک PDP یک ویژگی نمیتواند گفت آن ویژگی را مدل به صورت مفید در تصمیم گیری های خودش استفاده نمیکند.

زیر بخش دوم: Lime

اگر مدل ما در نقاط محلی به صورت یک آستانه عمل کند یا decision boundry ناپیسوستگی یا شکست داشته باشد. این موضوع باعث میشود هرچه قدر هم محلی بکنیم آن نقطه را نتوانیم با یک مدل خطی تفسیر و جایگزین کنیم چون ذات خطی ندارد حتی در نقاط همسایگی محلی کوچک آن.

وقتی یک مدل L-Lipshitz است با یک L محدود این باعث میشود عملا یعنی به ازای هر دو تا نقطه xی که داریم اختلاف آنها ضربدر یک ثابت L اختلاف نتایج تابع خروجی مدل نیز نزدیک و محدود باشد و این یعنی اگر تابع هدف مدل ما Lipshitz باشد عملا ما دیگر شکستگی یا ناپیسوستگی در مدل نخواهیم داشت و همین موضوع سبب میشود که عملا بتوان به صورت local یک تابع خطی روی همسایگی محلی فضای نهان مدلی خطی را فیت بکنیم.

$$|f(x_1) - f(x_2)| \leq K|x_1 - x_2|.$$

شکل 4. رابطه L-Lipshitz

زیر بخش سوم: Shapley Values

الف) در این بخش اگر دقت کنیم طبق شکل 5 رابطه ای که داریم برای محاسبه shapley value تمام ویژگی ها است. حال ما صرفا برای ویژگی زام میخواهیم.

$$f(x^i) - \mathbb{E}[f(X)]$$

شکل 5. رابطه محاسبه shapley value برای تمام ویژگی ها

در ابتدا رابطه را کمی میخواهیم با استفاده از classifier فی که در اختیار داریم ساده کنیم. سمت چپ منها که عملا خود رابطه شکل 6 میشود برای نمونه خاص i. سمت راست منها هم از آنجایی که expectation یک تبدیل خطی است صرفا روی تک تک x ها اعمال میشود و beta از داخل آن بیرون می آید.

$$f(x) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

شکل 6. Classifier f(x) در این مسئله

در نهایت ما به رابطه زیر (شکل 7) میرسیم.

$$f(x^i) - \mathbb{E}[f(X)] = \left(\beta_0 + \sum_{j=1}^p \beta_j x_j^i \right) - \left(\beta_0 + \sum_{j=1}^p \beta_j \mathbb{E}[X_j] \right)$$

شکل 7. رابطه نهایی محاسبه shapley value برای f خاص ما (برگرفته از deepseek)

حال با جایگذاری j دلخواه ما در این رابطه از آنجایی که بتا صفر که خنثی میشود بقیه ترم ها هم چون j نیستند و عملاً ما j Shapley value را میخواهیم حساب بکنیم در رابطه نهایی ظاهر نمیشوند و در نهایت چیزی که باقی میماند عبارتی است که میخواستیم آن را اثبات بکنیم. (شکل 8)

$$\beta_j (x_j^i - \mathbb{E}[X_j])$$

شکل 8. Shapley value ویژگی j برای مدل f

ب) طبق اینکه دیدیم shapley value برای مدل های خطی بسیار ساده و نیاز به محاسبات نمایی ندارد و بلکه با یک رابطه ساده بدست می آید. میتوانیم ما مدل های پیچیده black-box خود را با یک مدل خطی محلی تقریب (local linear surrogate model) بزنیم. و سپس بیایم و روی این مدل تقریب زده شده که خطی است و ضرایب آن را داریم برای یک ویژگی و نمونه خاص shapley value آن را به راحتی و بدون استفاده از اردر نمایی بدست آوریم و محاسبات را بشدت کاهش دهیم و تفسیر پذیری لازم را با تقریب مناسب بزنیم.

زیربخش چهارم: NAM

معماری این روش به این صورت است که به ازای هر featureی که ما در دیتای tabular داریم یک مدل DNN آموزش میدهد و خرجی نهایی میشود جمع خروجی تمامی مدل ها به ازای ورودی ویژگی متناظر آن که از مرتبط با دیتای ورودی است. همانطور که در شکل 9 مشاهده میشود خروجی مدل NAM میشود جمع خروجی های متناظر ویژگی های ورودی که هر کدام از این مدل ها که با f_i نمایش داده میشوند یک Deep Neural Network هستند و تنها فرق آنها با شبکه های عمیق معمولی آن است که برای اینکه بتوانند حالت های jagged را نیز مدل بکنند که خیلی smooth نیستند از یک hidden layer متفاوتی از حالت عادی که با استفاده از Relu است که در شکل 9 آمده استفاده میکنند و بلکه از hidden layerی به اسم ExU استفاده میکنند (شکل 10) تا به خاطر وجود عبارت نمایی e به توان w بتوان تغییرات ریز که نیاز به یک افزایش یا کاهش زیاد است را مدل بکند.

$$h(x) = \text{ReLU}(w \cdot x + b)$$

شکل 9. Hidden layer در مدل عمیق عادی - relu

$$h(x) = f(e^w \cdot (x - b))$$

شکل 10. Hidden layer در مدل معرفی شده NAM در هر کدام از f ها - ExU

برای بدست آوردن تفسیر ویژگی نام در مدل صرفاً کافی است هر $f_i(x_i)$ برای هر x_i بی که در بازه x_i های valid هستند را محاسبه کنیم و سپس نمودار $f_i(x_i)$ را بر حسب x_i آن بکشیم. و این باعث میشود میزان اهمیت آن ویژگی در مدل را متوجه شویم.

این مدل مزیتی که نسبت به مدل خطی دارد این است که کامل میتواند پیچیدگی های داخل یک تسک را مدل بکند اما یک مدل خطی صرفاً decision boundry خطی دارد و این باعث میشود هر نوع تسکی را نتواند مدل بکند. دلیل اینکه میتواند تسک های پیچیده با الگو های پیچیده را مدل بکند به DNN و hidden layer های ExU آن برمیگردد.

محدودیت اصلی آن به این برمیگردد که چون هر ویژگی به تنهایی برای آن یک مدل طراحی میکند و آموزش میدهد عملاً رابطه های که بین ویژگی ها هستند به خوبی مدل نمیشود و صرفاً در انتها با همدیگر نتایج مدل های هر ویژگی با هم جمع میشود. که در کار های پیش رو پیشنهاد میکند از حالت های pair-wise نیز استفاده گردد.

NAM نیز مشکل اینکه داده های extrapolation استفاده میشوند را دارد زیرا ما همچنان درست که در بازه وجود x_i ها داریم به مدل مقدار میدهیم و آن را plot میکنیم اما میزان آمدن آن ویژگی در دیتاست هم احتمال رخ داد آن در نواحی مختلف متفاوت است. هم که ویژگی مدنظر ما در کنار ویژگی های دیگر وابستگی دارد و هر عددی در کنار هر ویژگی نمیشود. پس در کل این مشکل همچنان در این متد نیز وجود دارد و حل نشده است.

بخش دوم: سوالات پیاده سازی

در این بخش، روش LIME برای تفسیر تصمیم یک مدل Pre-trained Resnet18 در حوزه تصویر پیاده سازی می‌شود. هدف این است که بفهمیم چرا یک تصویر خاص در یک کلاس مشخص (مثلاً «گربه») دسته‌بندی شده است و مدل به کدام بخش های عکس توجه کرده و این نتیجه را گرفته است. برای این

کار، ابتدا تصویر ورودی به چندین بخش تقسیم می‌شود. سپس با حذف و تغییر تصادفی این بخش‌ها، نسخه‌های زیادی از تصویر ساخته می‌شود و مدل روی همه آن‌ها پیش‌بینی انجام می‌دهد. در نهایت، یک مدل خطی ساده روی این نمونه‌ها آموزش داده می‌شود تا بخش‌هایی را که بیشترین تأثیر را بر تصمیم مدل داشته‌اند، شناسایی کند.

زیربخش اول: مقداردهی مدل

در ابتدا وزن‌های مدل را لود کردم و به مدل Resnet18 دادم. سپس تصویر گربه را به مدل دادم. مدل چون pretrained است نیاز به آموزش آن نیست. مدل با **90.5% confidence** کلاس گربه را برای عکس گربه پیش‌بینی کرد.

```
Image: TAI_HW2_Files\Q1\code\cat.jpg  
Model prediction for class 282 (cat): 0.9055 (90.55%)
```

شکل 11. نتیجه احتمالی کلاس گربه توسط مدل

Input Image: cat.jpg
Model confidence for 'cat' (class 282): 90.55%

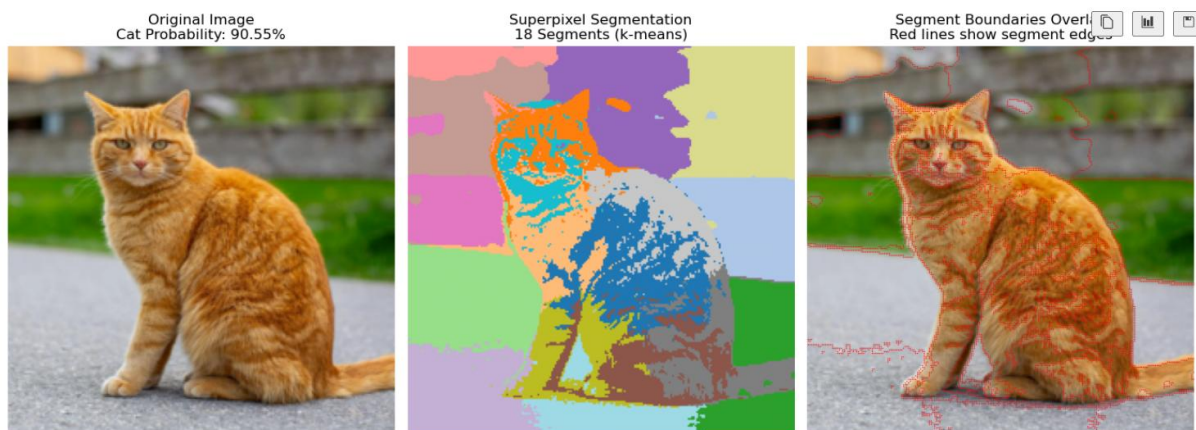


شکل 12. تصویر گربه داده شده به مدل

زیربخش دوم: بخش بندی تصویر

ما چندین الگوریتم مختلف برای segmentation داریم. در این کد از الگوریتم **K-means** استفاده شده است. تابع ابتدایی عکس و تعداد بخش ها و تعداد پیمایش های حداکثری را برای بدست آوردن خوشه ها ورودی میگیرد. الگوریتم k-means به صورت کلی به این صورت عمل میکند که می آید k نقطه (پیکسل) به صورت رندوم در فضای عکس انتخاب میکند (در اینجا 18 نقطه و به انها نقاط مرکزی میگوییم). سپس فاصله تمام نقاط (پیکسل ها) به آن را محاسبه میکند (این فاصله ترکیبی از فاصله مکانی و رنگی است). حال هرکدام از این 18 نقطه آن نقاطی که نزدیک ترین به نقطه مرکزی هستند را یک خوشه در نظر میگیرد و در نتیجه 18 خوشه تشکیل میشود. در نهایت برای هر خوشه نقطه مرکزی جدید ان را با توجه به میانگین فواصل نقاط درون آن بدست می آورد. یک دور پیمایش به طر کامل تمام شده و دوباره مراحل ذکر شده را از اول پیش میرود تا یا دیگر نقاط مرکزی بروزرسانی نشوند و یا به حداکثر تعداد پیمایش ها توسط تابع برسیم. به این شکل 18 خوشه ما در تصویر بدست می آید.

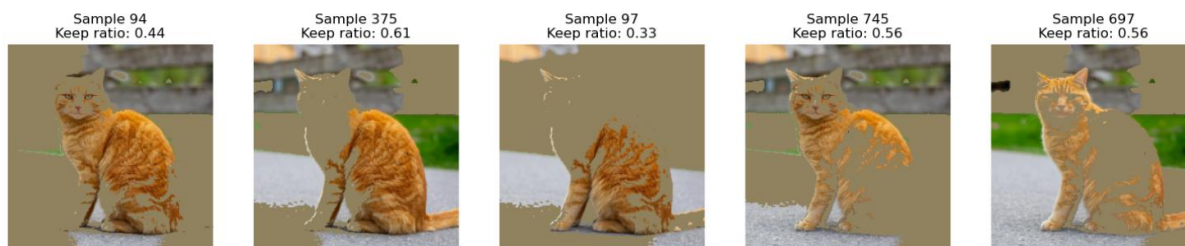
(The distance is Euclidean distance in 5D space (R,G,B,x,y))



شکل 13. تصویر اصلی = تصویر خوشه بندی شده - تصویر با مرز های خوشه بندی

زیربخش سوم: تولید نمونه های تغییر یافته

تمامی 1000 آرایه ی 18 بعدی که عملاً masked_array نامیده میشوند را درست کردیم به صورت رندوم. عکس های تغییر یافته را از این آرایه بدست آوردیم و انها را در پوشه processed_data/cat قرار دادیم. چند نمونه از این تصاویر در شکل 14 آمده است.



شکل 14. نمونه هایی از تصاویر تغییر یافته همراه با z ratio آنها

زیربخش چهارم: وزن دهی تصاویر تغییر یافته

دلیل انجام این وزن دهی به این خاطر است که نمونه هایی که خیلی تغییر میکنند و عملاً z ratio آنها کم است و عملاً خوشه های کمی از تصویر اصلی در تصویر تغییر یافته وجود دارد زیاد مبنای خوبی برای یادگیری مدل نیستند زیرا دیگر گربه ای به آن صورت در تصویر دیده نمیشود چه بسا حتی بخش های مهم و تاثیرگذاری آمده باشد. این موضوع باعث شده تا ما به نمونه های تغییر یافته ای که تعداد خوشه های اصلی بیشتری در آنها وجود دارد (z ratio بالاتر) را با وزن بیشتری بدهیم و آنها که کمتر هستند را وزن کمتری بدهیم. وزن های هر نمونه هم با رابطه ی آمده در شکل 15 محاسبه میشود.

$$w(z) = \exp\left(-\frac{[\text{dist}(z)]^2}{\sigma^2}\right)$$

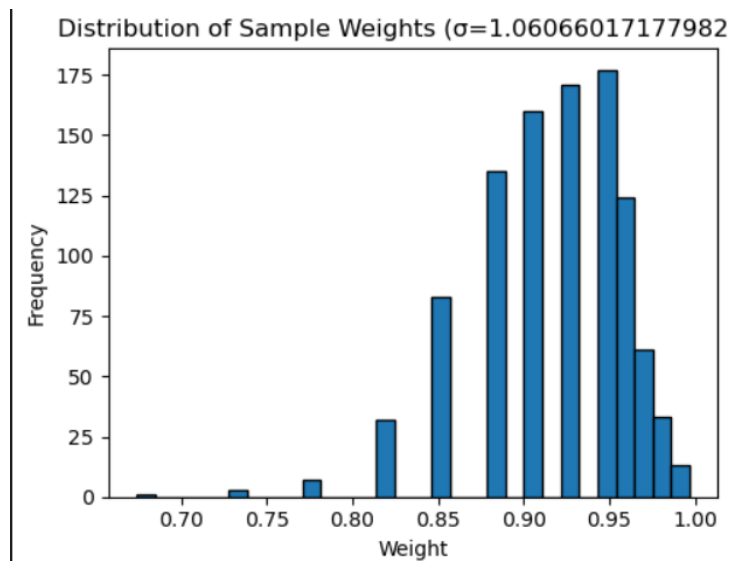
شکل 15. رابطه بدست آوردن وزن هر نمونه با توجه به آرایه z (masked) آن

$$\text{dist}(z) = 1 - \frac{z \cdot 1}{|z| |1| + \varepsilon}$$

شکل 16. محاسبه distance z

$$\sigma = 0.25 \times \sqrt{d}$$

شکل 17. محاسبه انحراف معیار برحسب تعداد کل خوشه ها



شکل 18. توزیع وزن نمونه ها

زیربخش پنجم: آموزش مدل خطی

استفاده از **L1-Regularization** باعث میشود که مدل ما وزن هایی که کم هستند مقدار مطلقشان را به سوی 0 سوق دهد. از انجایی که انسان خود در پیشبینی کردن به چند ویژگی اصلی و مهم توجه میکند و بقیه ویژگی ها را کنار میگذارد ما هم میخواهیم مدل ما به ویژگی های اصلی که باعث پیشبینی آن میشود توجه کند و به همین خاطر است از این نوع regularization استفاده میکنیم تا خیلی از ویژگی ها وزن 0 پیدا کنند و باقی تصمیم گیرنده اصلی در پیشبینی مدل باشند.

حال دلیل اینکه L1 نسبت به L2 این امر را ممکن میکند این است که ما وقتی بروز رسانی میکنیم گرادیان میگیریم و مشتق L1 میشود عدد ثابت 1 و مشتق L2 میشود 2β و اگر دقت کنیم به ازای β ها کوچک تغییرات ما هم کوچک میشود (2β اما در L1 این طور نیست حتی اگر β تا کوچکی هم داشته باشیم به اندازه 1 ما تغییر خواهیم داشت. که این امر باعث میشود β های مقدار کوچک به صفر بروند.

ضرایب وزن ها به صورت heatmap در شکل 19 آمده است. همانطور که مشاهده میکنید مدل ما به درستی توانسته resnet را تفسیر کند و عملاً میگوید resnet با نگاه کردن و توجه کردن به خود گربه و نه background آن توانسته کلاس گربه بودن این عکس را تشخیص دهد.

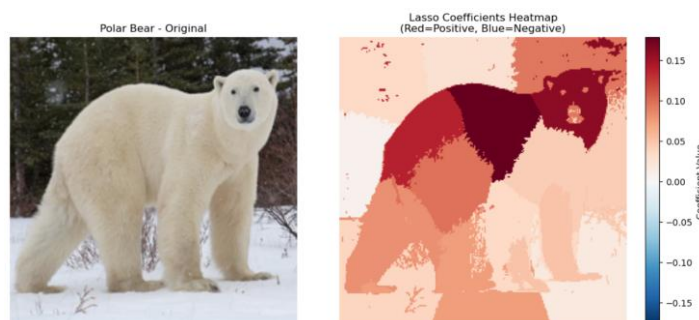


شکل 19. میزان توجه مدل به خوشه های مختلف گربه به صورت heatmap

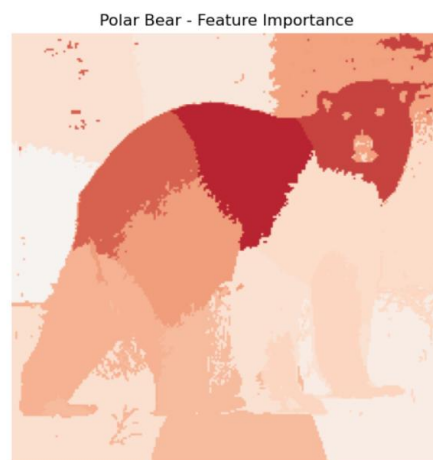
زیربخش ششم: تصویر خرس قطبی

دقیقا همان pipeline یی که برای عکس گربه داشتیم را برای عکس خرس قطبی اعمال میکنیم (از انجایی که این روش LIME یکی روش تفسیر پذیری Local است به ازای هر عکس/نمونه باید خوشه بندی مجدد انجام شود و کل مدل خطی دوباره آموزش ببیند).

نتیجه نهایی روی خرس قطبی را در شکل 20 مشاهده میکنیم. همانطور که میبینیم این دفعه مدل ما به پس زمینه تصویر اهمیت داده است با اینکه بهتر است مدل Restnet تنها به خود تصویر خرس اهمیت بدهد. اما دلیل اینکه این مورد این شکلی شده است آن است که دیتای که این مدل Resnet دیده است برای خرس قطبی تقریبا همه خرس قطبی ها در قطب ها هستند و این باعث میشود تمامی عکس های آن با پس زمینه سفید و برفی باشد و خود به خود مدل یاد میگیرد انگار آن هم جزوی از خرس قطبی بودن است و این خود یعنی مدل از ویژگی های بایاس دار یاد گرفته است و عملا با این تفسیر متوجه میشویم مدل میتواند آموزش بهتر و کم بایاس تری هم داشته باشد که در حال حاضر ندارد.



شکل 20. میزان وزن های خوشه های مختلف نمایش داده شده به صورت heatmap



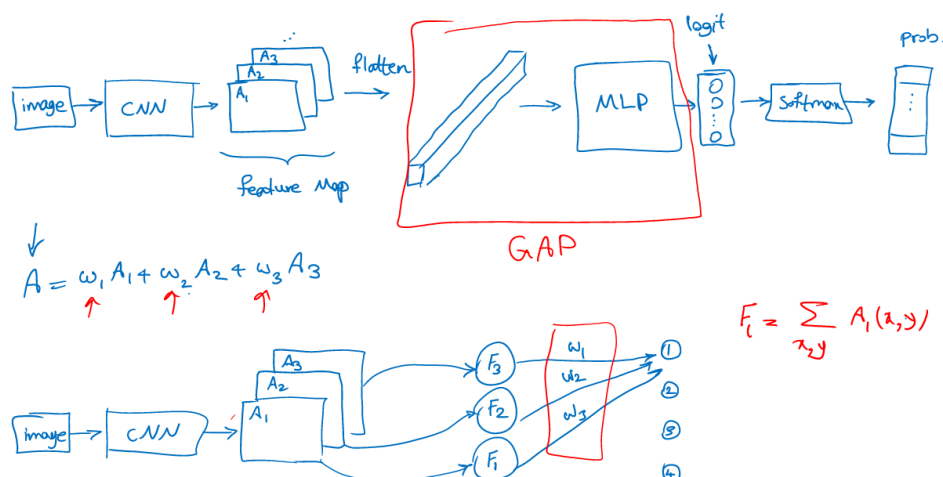
شکل 21. مقایسه بین heatmap توجه مدل به بخش های مختلف گربه با خرس قطبی

سوال دوم: توصیف پذیری مدل های هوش مصنوعی با استفاده -Grad-CAM

بخش اول: Grad-CAM

زیربخش اول: پرسش و پاسخ

سوال 1) Grad-CAM یک مدل تعمیم یافته از CAM است. مدل CAM در شکل 22 آمده است. این مدل برای یکسری معماری CNN های خاص جوابگو است. همانطور که در شکل دیده میشود در صورتی که بعد از لایه های CNN feature extraction مورد نظر و پس از رد کردن این فیلتر ها که feature map ها را به وجود می آورند در صورتی که یک mlp داشته باشیم با جایگزین کردن آن با مدل CAM و انجام global average pooling و سپس آموزش دادن این مدل جدید و بدست آوردن وزن ها میتوانیم همین وزن ها را در feature map ها ضرب کنیم و در نهایت اهمیت هریک از پیکسل های عکس را صرفاً در تسک classification و با این نوع خاص معماری CNN بدست آوریم.



شکل 22. معماری CAM

در روش Grad-CAM دیگر نه این معماری خاص مورد نیاز است و هر نوع معماری CNN یی داشته باشیم میتوانیم این روش را روی آن اعمال کنیم همچنین دیگر فقط برای طبقه بندی استفاده نمیشود بلکه علاوه بر **Image Classification** برای **Visual Question Answering** و **Image Captioning** هم استفاده میشود. در روش Grad-CAM ما تنها نیاز به گرادیان های لایه آخر conv layer داریم که با استفاده از آن و بدون آموزش مجدد مدل ضرایب feature map ها را بدست آوریم و با انجام جمع به روی آنها در نهایت بگوییم مدل به کجا عکس برای یک کلاس خاص توجه کرده است.

$$\alpha_k^c = \overbrace{\frac{1}{Z} \sum_i \sum_j}^{\text{global average pooling}} \underbrace{\frac{\partial y^c}{\partial A_{ij}^k}}_{\text{gradients via backprop}}$$

شکل 23. رابطه دقیق ضرایب روش Grad-CAM

اثبات این موضوع در شکل 23 آمده است. همانطور که دیده میشود به جای محاسبه w_i ها به روش معماری خاص CAM میتوان آن را با روش جدید بدست آورد که دقیقاً همان عدد قبلی است اما در این روش نیاز نیست بخشی از مدل را جایگزین کنیم و با گرادین های لایه آخر convolution layer میتوانیم به خواسته خود برسیم (دلیل استفاده از لایه آخر conv هم آن است که به خوبی هم detailed spatial information را نظر میگیرد و هم high-level semantics را به خوبی مدل میکند). این موضوع باعث میشود بتوان این روش تفسیر پذیری را روی تمامی مدل های CNN و هرنوع تسکی اعمال کرد.

Let us define F^k to be the global average pooled output,

$$F^k = \frac{1}{Z} \sum_i \sum_j A_{ij}^k \quad (4) \quad \text{Summing both sides of (8) over all pixels } (i, j),$$

CAM computes the final scores by,

$$Y^c = \sum_k w_k^c \cdot F^k \quad (5) \quad \sum_i \sum_j w_k^c = \sum_i \sum_j Z \cdot \frac{\partial Y^c}{\partial A_{ij}^k} \quad (9)$$

where w_k^c is the weight connecting the k^{th} feature map with the c^{th} class. Taking the gradient of the score for class c (Y^c) with respect to the feature map F^k we get,

Since Z and w_k^c do not depend on (i, j) , rewriting this as

$$\frac{\partial Y^c}{\partial F^k} = \frac{\frac{\partial Y^c}{\partial A_{ij}^k}}{\frac{\partial F^k}{\partial A_{ij}^k}} \quad (6) \quad Z w_k^c = Z \sum_i \sum_j \frac{\partial Y^c}{\partial A_{ij}^k} \quad (10)$$

Taking partial derivative of (4) w.r.t. A_{ij}^k , we can see that $\frac{\partial F^k}{\partial A_{ij}^k} = \frac{1}{Z}$. Substituting this in (6), we get,

Note that Z is the number of pixels in the feature map (or $Z = \sum_i \sum_j 1$). Thus, we can re-order terms and see that

$$\frac{\partial Y^c}{\partial F^k} = \frac{\partial Y^c}{\partial A_{ij}^k} \cdot Z \quad (7)$$

From (5) we get that, $\frac{\partial Y^c}{\partial F^k} = w_k^c$. Hence,

$$w_k^c = Z \cdot \frac{\partial Y^c}{\partial A_{ij}^k} \quad (8) \quad w_k^c = \sum_i \sum_j \frac{\partial Y^c}{\partial A_{ij}^k} \quad (11)$$

شکل 23. اثبات برابر بودن وزن های استفاده شده در CAM با ضرایب آلفا استفاده شده در Grad-CAM

سوال 2) دلیل استفاده از تابع **ReLU** در ترکیب نهایی نقشه های فعالسازی آن است که این تابع صرفاً مقادیر مثبت را رد میکند و تمامی مقادیر منفی را مقدار 0 می دهد. این امر موجب میشود تنها به ویژگی ها و پیکسل هایی اهمیت بدهیم که در حال کمک به صورت یک نقش مثبت برای پیدا کردن

کلاس مورد نظر هستند و دیگر پیکسل هایی که بیشتر احتمالاً مربوط به کلاس های دیگر هستند در حال کاهش احتمال این کلاس ما میشوند را در نظر نمیگیریم تا در نهایت صرفاً آن پیکسل هایی که تاثیر مثبت در پیشبینی داشتند را در خروجی ببینیم و نه انهایی که در حال مانع شدن برای پیشبینی کلاس مورد نظر بودند.

در صورت حذف ReLU در heatmap نهایی ما پیکسل هایی که در حال مقاومت برای این کلاس هستند هم دیده خواهد شد در صورتی که زیاد معنایی ندارند. همچنین نبود آن باعث میشود localization مورد نظر به خوبی نمایش داده نشود. این یعنی گرادیان های مثبت یعنی احتمال کلاس مورد نظر در حال افزایش و گرادیان های منفی یعنی احتمال کلاس مورد نظر ما را کاهش میدهند.

سوال 3) این کشف با آزمایش و آموزش مدل روی یک دیتاست که تصاویری از پرستارها و دکترها داشت انجام شد. دیده شد که چون تعداد دکترهای مرد زیاد (حدود 78٪) و تعداد پرستارهای خانم زیاد (93٪) بودند مدل به صورت بایاس دار روی ویژگی جنسیت با دیدن موهای شخص و تمرکز بر جنسیت آموزش دیده بود. که مدل Grad-CAM ما با هایلایت بخش های موهای عکس های داده شده به مدل به ما این موضوع را فهماند و ما متوجه شدیم مدل به صورت بایاس دار آموزش دیده است. سری بعدی مدلی با آموزش غیر بایاس دار که در واقع صرفاً با زیاد کردن داده های جنس مخالف در هر کلاس انجام شد و پس از آن با تحلیل Grad-CAM میتوان دید که مدل به بخش هایی مانند ابزارالات پزشکی و روپوش توجه کرده تا جنسیت (موی سر شخص).

امروزه از مدل های هوش مصنوعی در تصمیم گیری های مهمی استفاده میشود و به لحاظ اخلاق ما باید fairness مدل را مطمئن بشویم تا به صورت بایاس دار چیزی را جاب انسان ها پیشبینی نکند که سرنوشت ها عوض شود و این کار با مدلی مثل Grad-CAM امکان پذیر میشود.

سوال 4) در حالی که Grad-CAM قابل تمایز بین کلاس ها (class-discriminative) است و نواحی مرتبط با تصویر را محلی سازی می کند، اما توانایی برجسته سازی جزئیات ریز (fine-grained details) را مانند روش های تجسم گرادیان در فضای پیکسل (مانند Guided Backpropagation و Deconvolution) ندارد.

به عبارت دیگر

- **Grad-CAM** درواقع **region-level** است.

- **Guided Backpropagation** درواقع **pixel-level** است.

پس با ترکیب این دو که عملاً با ضرب کردن پیکسلی آنها را ترکیب میکنیم می‌توان هم دقت پیکسلی Guided Backpropagation را داشت و هم قابلیت تفکیک کلاسی Grad-CAM را.

زیربخش دوم: پیاده سازی

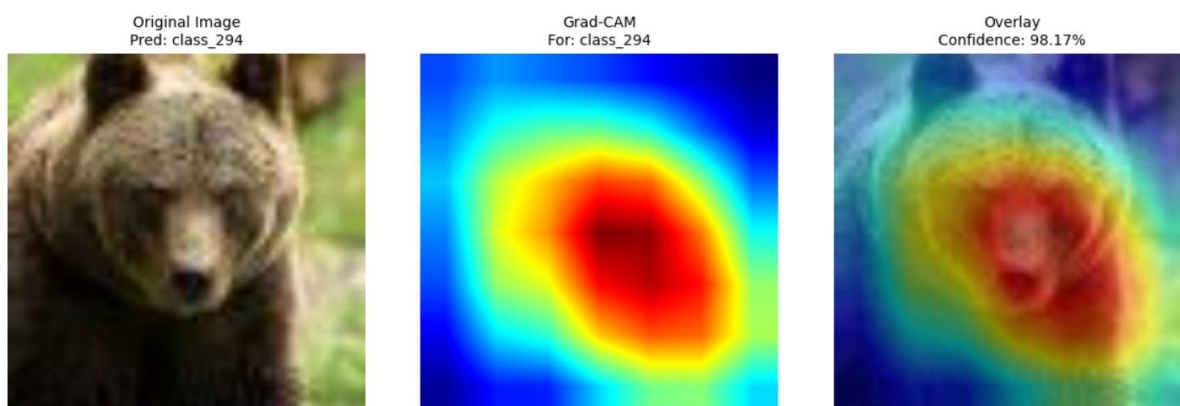
الف) مدل Resnet18 که بروی دیتای ImageNet-1k آموزش دیده شده است را از طریق پوشه های داده شده که وزن های مدل در آن فایل pth است لود کردم.

ب) سپس پس از دانلود دیتاست TinyImageNet که لینک آن در داک فایل پروژه داده شده است داخل پوشه دیتاست در دایرکتوری قرار دادم و در کد آن را لود کردم و به دیتا لودر دادم.

ج) با استفاده از torch به صورت کامل Grad-CAM را پیاده سازی نمودم. راجب گرفتن Inference درست و نشان دادن saliency map ها باید این نکته را متذکر مدل pretrained عملاً روی 1000 کلاس آموزش داده شده است (ImageNet-1k دیتاست آن بوده است). اما دیتاست داده شده در صورت پروژه که بنده دانلود کردم توسط لینکی که گذاشته شده بود برای tinyImageNet است که در واقع 200 کلاس دارد. راه حلی که استفاده کردم آن است که بیاییم کلاس های مشترکی که در جفت این دو دیتاست وجود دارد را درنظر بگیریم و روی آنها پیمینی را انجام دهیم و آن هایی که درست پیمینی میشوند را نمایش دهیم.

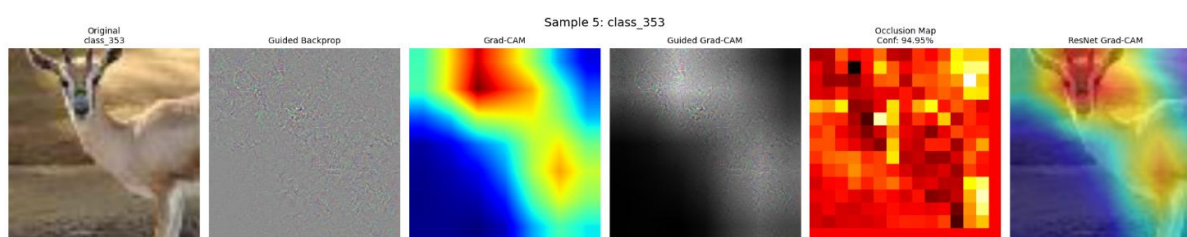
این راه حل هم با چالش رو به رو شد زیرا به علت نبود label های دقیق اسمی هرکلاس و صرفاً به صورت class_i بودن label نمیتوانیم به اسم دقیق آنها دسترسی داشته باشیم تا ست یی از کلاس های مشترک بدست آوریم و ایدی ها هم یکسان نیستند. یک راه حل آن است که از نت به صورت on the fly fetch بکنیم اسمی کلاس ها را که به علت قطعی اینترنت بین الملل نتوانستیم این اطلاعات را دریافت کنیم. به همین خاطر راه حلی که در نهایت در پیش گرفتیم آن است که به جای آن که با چک کردن برچسب بیاییم و درستی یک پیمینی را به دست آوریم گفتم آنهایی که confidence بالایی مدل پیمینی میکند را احتمالاً درست پیمینی میکند (این یک فرض غلط است اما به صورت احتمالی بالاتری

دارد که واقعا درست پیشبینی کرده باشد.) و آنها را نمایش دادیم اطلاعات مورد نظر خواسته شده آنها را. نتایج در شکل 24 دیده میشود.



شکل 24. Saliency map , overlay یک نمونه خرس با استفاده از Grad-CAM

د) به همین صورت مانند بخش ج این بخش را هم پیش بردیم و موارد خواسته شده را نمایش دادیم که در شکل 25 میتوان آن را دید.



شکل 25. یک نمونه آهو از 5 نمونه که با استفاده از Guided-backprop – Grad-CaM – Guided-Grad-CAM

بخش دوم: FinerGrad-CAM

زیربخش اول: پرسش و پاسخ

سوال 1) بله Grad-CAM در حال توضیح میزان اهمیت هر پیکسل در طبقه بندی آن کلاس مورد نظر ما است و در توضیح این موضوع به خوبی ("چگونگی") عمل میکند. اما طبق گفته مقاله این آن چیزی نیست ("چه چیزی") که کاربرد فراوان داشته باشد زیرا در صورتی که همان عکس را saliency map آن را با استفاده از این روش برای کلاسی دیگر اما مشابه به کلاس قبلی مان مدل رسم کند، میبینیم که بسیار شبیه saliency mapی است که در کلاس درست قبلی رسم شده است. همین موضوع یعنی discriminative localization بین کلاس ها مشابه به خوبی انجام نشده است. در تفسیر مدل ها هم ما

بیشتر به دنبال تفاوت ها و دلیل انتخاب یک کلاس نسبت به کلاس مشابه دیگر آن هستیم. با این دید نویسندگان مقاله به مسعله نگاه کردند و روش *FinerGrad-CAM* را پیشنهاد دادند.

سوال (2) مغز انسان وقتی میخواهد بین دو کلاس مشابه انتخاب کند که عکسی مربوط به کدام است بیشتر از اینکه هر کدام چه ویژگی دارد به آن که چه چیزی در بین این دو عکس تفاوت میکند دقت و توجه میکند. که همین توجه به تفاوت ها را مقاله با استعاره *spot the difference* بیان میکند.

در سطح ریاضی نیز مدل *FinerGrad-CAM* به این صورت پیاده سازی میشود که دقیقا همان رابطه ای که برای *Grad-CAM* ما داشتیم بیایم و گرادیان کلاس مشابه دیگری که به آن *reference class* میگوییم هم محاسبه کنیم و اعمال ضربی به آن از مقدار گرادیان کلاس اصلی کم کنیم تا مدل به جای اهمیت به ویژگی هایی در *feature map* که مختص همان کلاس هستند به میزان تفاوت این ویژگی ها در کلاس اصلی و کلاس رفرنس که مشابه این کلاس است دقت کند. رابطه ریاضی دقیق این مدل در شکل 24 آمده است.

$$\alpha_k^{c,d} = \frac{1}{Z} \sum_i \sum_j \frac{\partial(y^c - \gamma \times y^d)}{\partial A_k^{ij}},$$

شکل 24. ضرایب مدل *FinerGrad-CAM* به این صورت بدست می آیند - *d* کلاس رفرنس و *c* کلاس مورد نظر برای تفسیر است

سوال (3) ضریب گاما در رابطه هرچه به صفر نزدیک تر باشد یعنی به *baseline Grad-CAM* در حال نزدیک تر شدن هستیم و این یعنی به "نمای کلی شی" بیشتر در حال توجه کردن هستیم. هرچه هایپرپارامتر گاما بیشتر باشد یعنی به "جزئیات ریز تمایزدهنده" در حال توجه است.

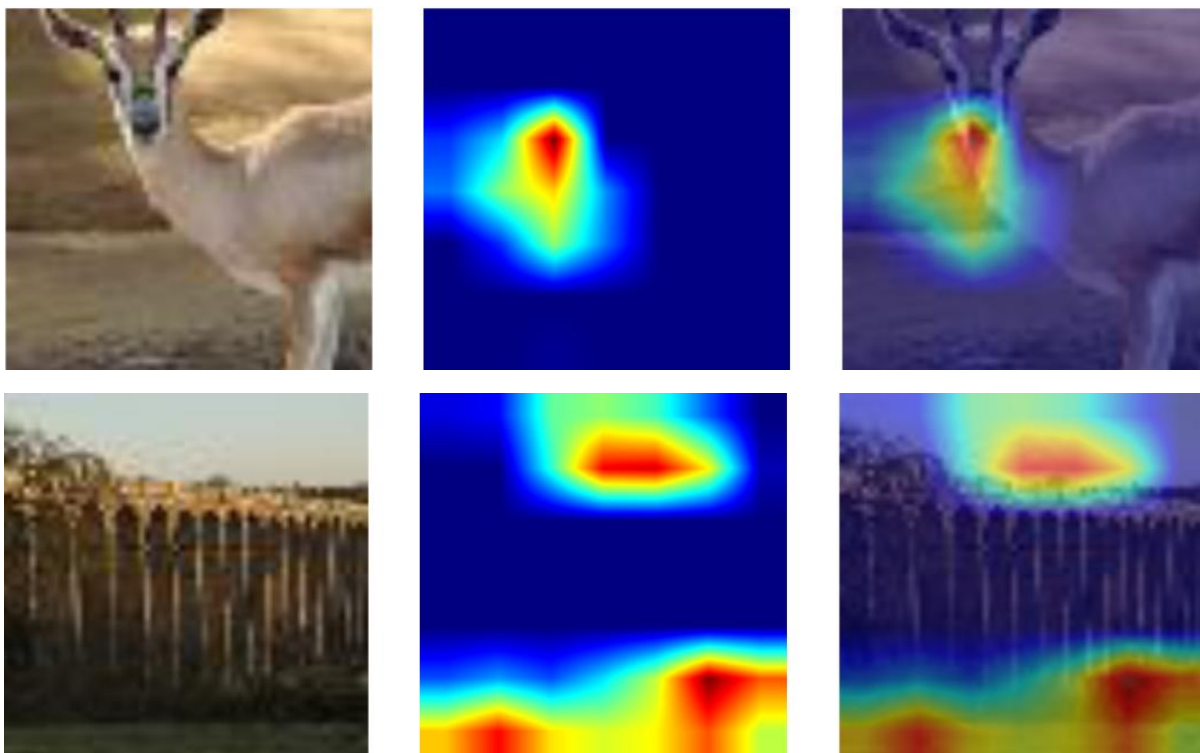
مثلا در زمینه ای مثل زیست شناسی و تشخیص پرندگان بسیار این پرندگان به هم شبیه هستند و میزان تمیز دهنده این پرندگان از هم است که برای کاربر و استفاده کننده از مدل مهم است که در این شرایط گاما را بیشتر میگذاریم تا علل انتخاب یک کلاس نسبت به کلاس یا کلاس های دیگر را بسنجد.

سوال (4) دقیقا چیزی که بررسی میکند آیا مدل ما دقیقا *key trait* ها را به درستی تشخیص میدهد یا خیر. این کار را با مقایسه *classification saliency map* و *attribute labels* انجام میدهد.

دلیل مهم بودن این موضوع هم آن است که مطمئن شویم مدل به بخش های درستی در حال توجه است و به ما جواب میدهد و مورد دیگر آن این است که این مدل بایاس دار نباشد.

زیربخش دوم: پیاده سازی

مقدار گاما را 3 گذاشتم که ضریب تعادل بین تفاوت سنجی پیدا کردن ویژگی های محلی باعث پیشبینی است و این عدد یک حد تعادل خوبی است. تعداد کلاس های رفرنس را نیز طبق گفته صورت سوال 3 گرفتم که باعث شود به 3 کلاس پر احتمال به جز کلاس اصلی هم توجه شود و تفاوت آنها را حساب کند. تمامی مپ ها نمایش داده شده در شکل های زیر هستند. (دقت کنید برخی از آنها به علت اینکه گاما 3 برای آنها زیاد بود و تصاویر کامل تک رنگ میشدند برای برخی تصاویر گاما را به 1.5 کاهش دادیم تا به صورت معنادار saliency map کشیده شود)



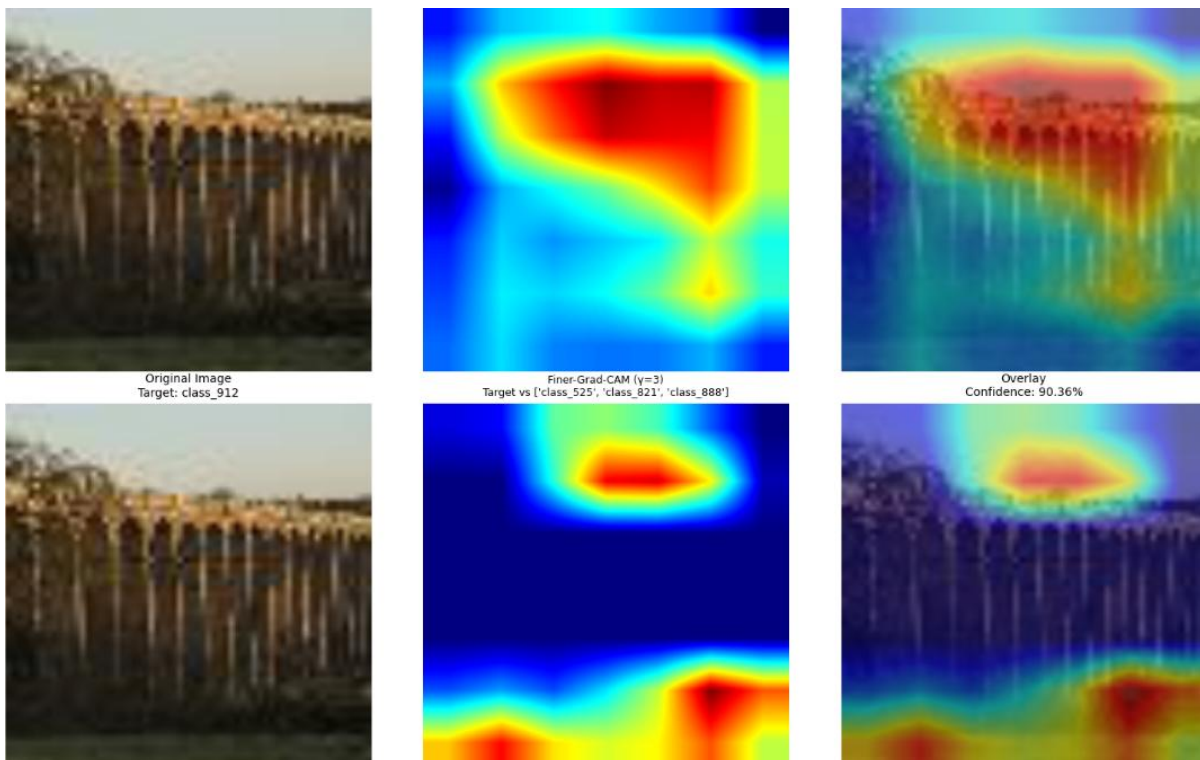
شکل 25. نمایش برخی از FinerGrad CAM نتایج

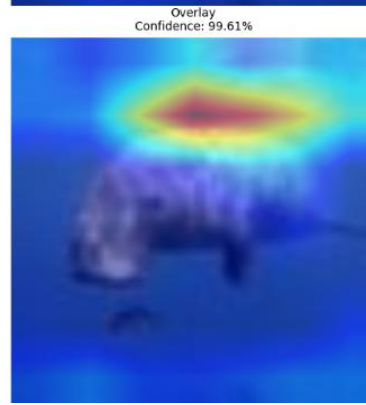
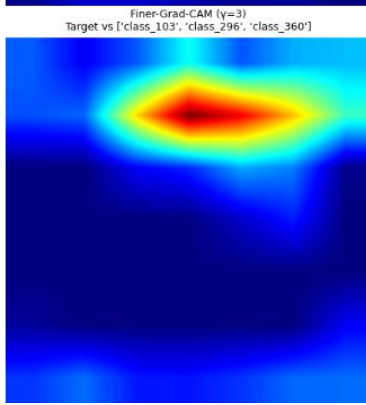
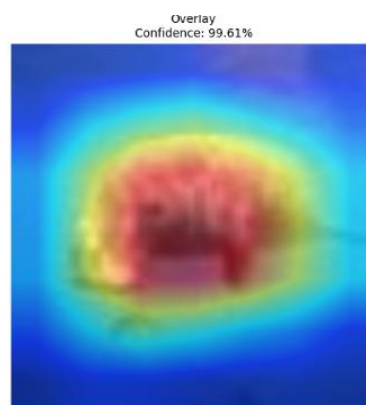
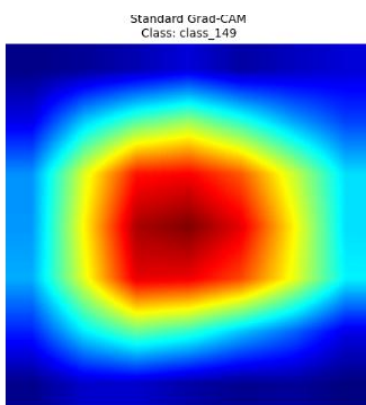
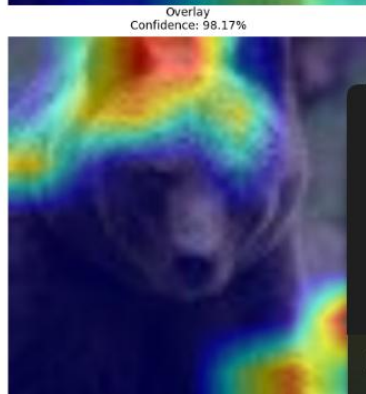
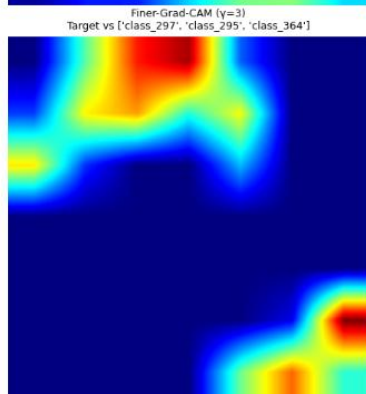
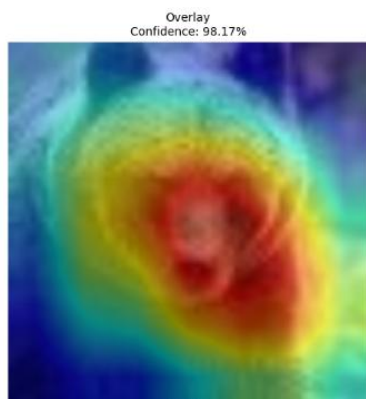
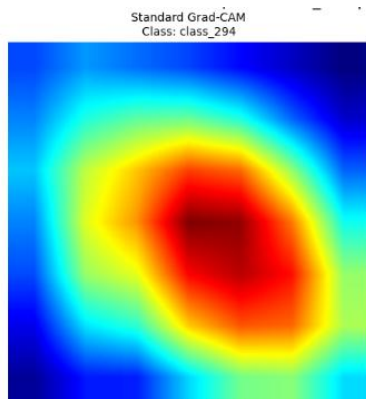
باقی در خود نوتبوک موجود هستند.

بخش سوم: مقایسه

زیربخش اول: مقایسه دو روش

برای مقایسه شکل های زیر وجود دارد که هم grad-CAM و هم FinerGrad-CAM نمایش داده شده اند نتایج آنها.





باقی تصاویر در نوت بوک آمده اند.

به صورت کلی در Grad-cam ما کل تصویر که کلاس ما را تشکیل میدهد را در saliency map مشاهده میکنیم اما در FinerGrad-CAM ما بخش های جزعی از آن جسم را که صرفا مختص همان عکس است و آنها باعث تفاوت آن با باقی موجودات میشوند را مشاهده میکنیم بر فرض مثال در تصویر موجود دریی اگر دقت کنیم در حالت grad-CAM به کل موجود دریایی توجه کرده اما در Finergrad-CAM صرفا به بخش از بال بالایی آن که ان را منحصر به فرد میکند نسبت به باقی موجودا دریایی توجه کرده است.

همچنین در خرس هم اگر دقت کنیم به گوش های آن توجه کرده که احتمالا این ویژگی است که از کلاس هایی که احتمال داشت آنها را پیشبینی بکند متمایز میسازد.

یک نکته دیگر که هم باید اشاره کنم ان است که به دلیل اینکه طبق صحبت هایی که در پیش کردیم برچسب های درست را برای عکس ها نداشتیم و از روی میزان اطمینان مدل درستی را حدس زدیم و خود این موضوع باعث میشود کلاس های دیگر احتمال کمی داشته باشند و کمی موضوع مقایسه کلاس بیشترین احتمال با 3 کلاس رفرنس دیگر را در finergrad-cam با مشکل موجه کند و برای همین هم تصویر خیلی ایده ال بدست نیامدند اما باز هم حتی در این تصاویر هم تفاوت عملکرد finergradcam با gradcam به وضوح قابل مشاهده است.

- [1] Agarwal, R., Frosst, N., Zhang, X., Melnick, L., Lengerich, B., Caruana, R., & Hinton, G. E. (2021). Neural Additive Models: Interpretable Machine Learning with Neural Nets. *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS 2021)*.
- [2] Apley, D. W., & Zhu, J. (2019). Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models. *arXiv preprint arXiv:1612.08468v2*.
- [3] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2019). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *arXiv preprint arXiv:1610.02391v4*.
- [4] Zhang, Z., Gu, J., Chowdhury, A., Mai, Z., Carlyn, D., Berger-Wolf, T., Su, Y., & Chao, W.-L. (2025). Finer-CAM: Spotting the Difference Reveals Finer Details for Visual Explanation. *arXiv preprint arXiv:2501.11309v2*.